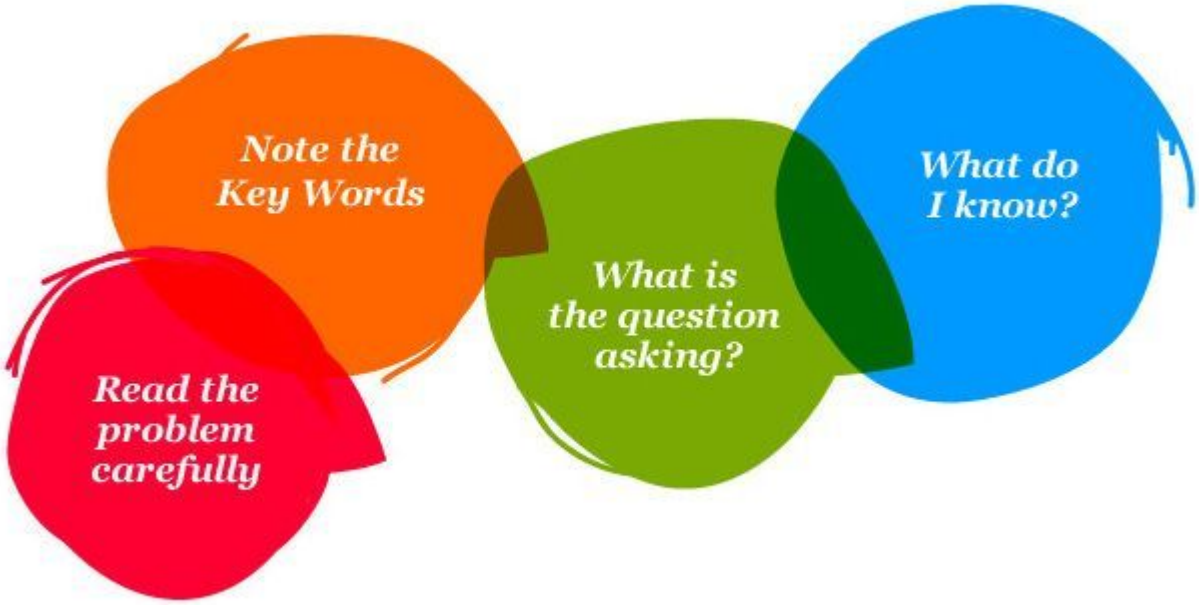
CS 103000
Prof. Madeline Blount

Week 9:
FUNCTIONS + ABSTRACTION



Dall-E 2: cats learning C++ in the forest on ‘90’s technology


WORKS
ON MY MACHINE
imgflip.com


Note the
Key Words
What do
I know?
What is
the question
asking?
Read the
problem
carefully

Programming is like writing a book...
Except when you miss a single comma on page 126
the whole thing makes no sense.

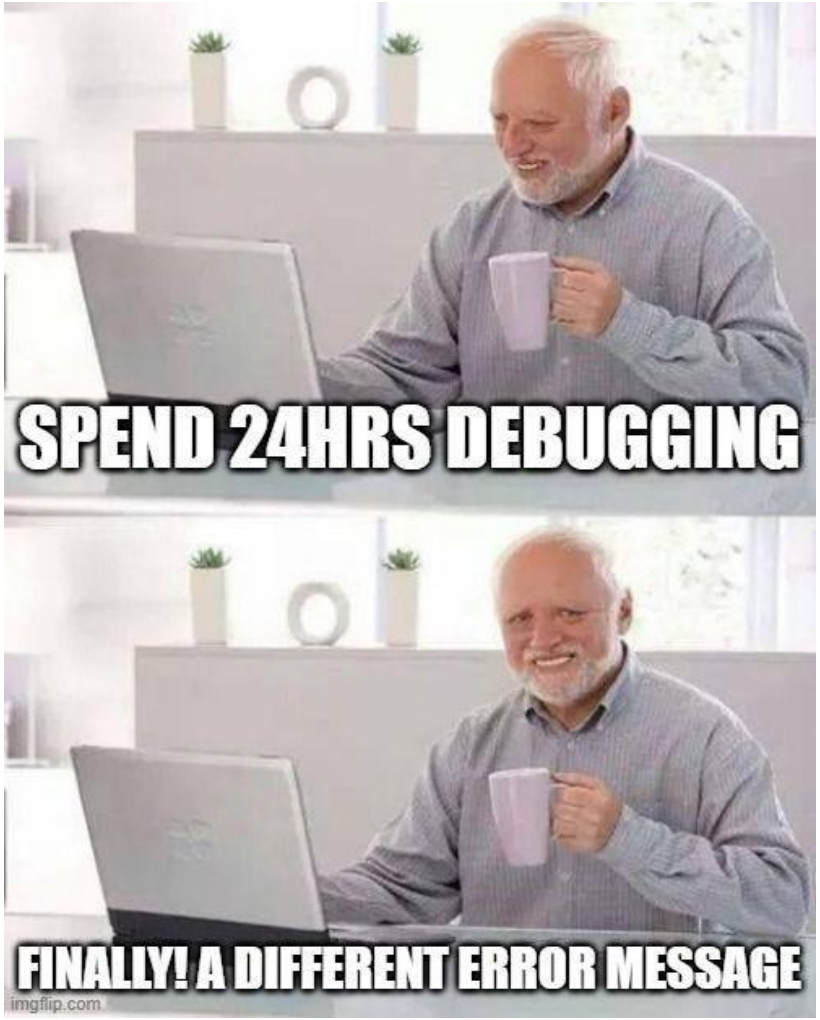
Microsoft

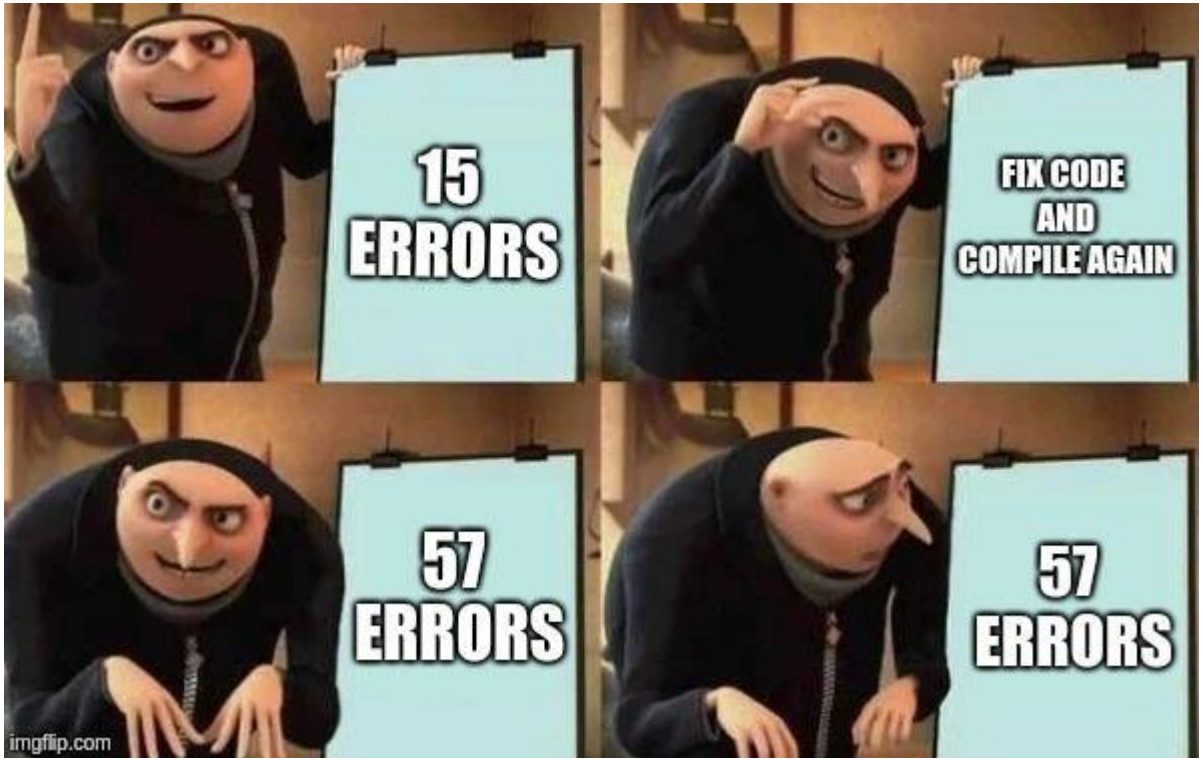
my program: *works perfectly*

me: *cleans up the code*

also my program:




SPEND 24HRS DEBUGGING
FINALLY! A DIFFERENT ERROR MESSAGE
imgflip.com


15
ERRORS
FIX CODE
AND
COMPILE AGAIN
57
ERRORS
57
ERRORS
imgflip.com

Error on line 42

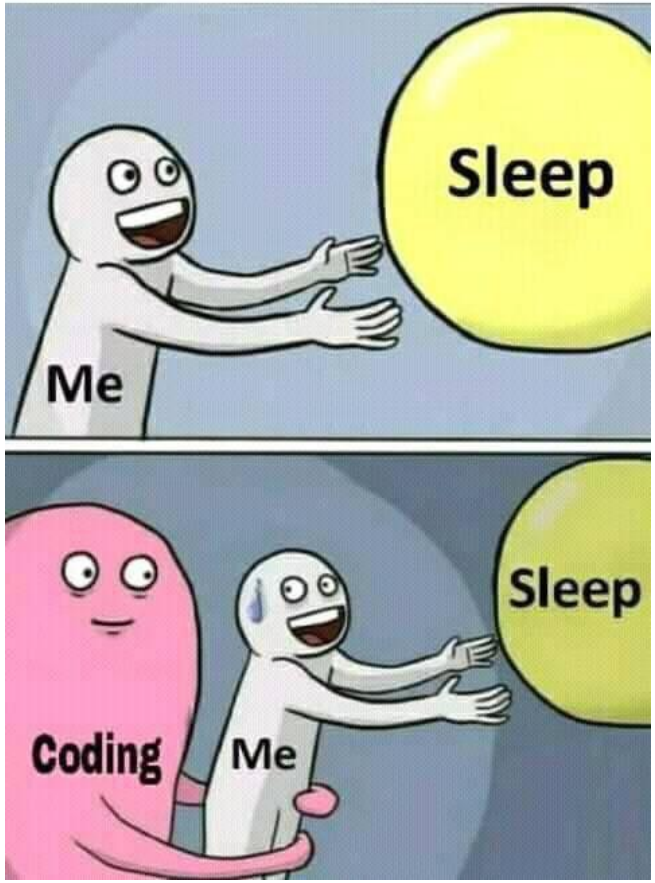


```
41    });
42
43    if (includ
```



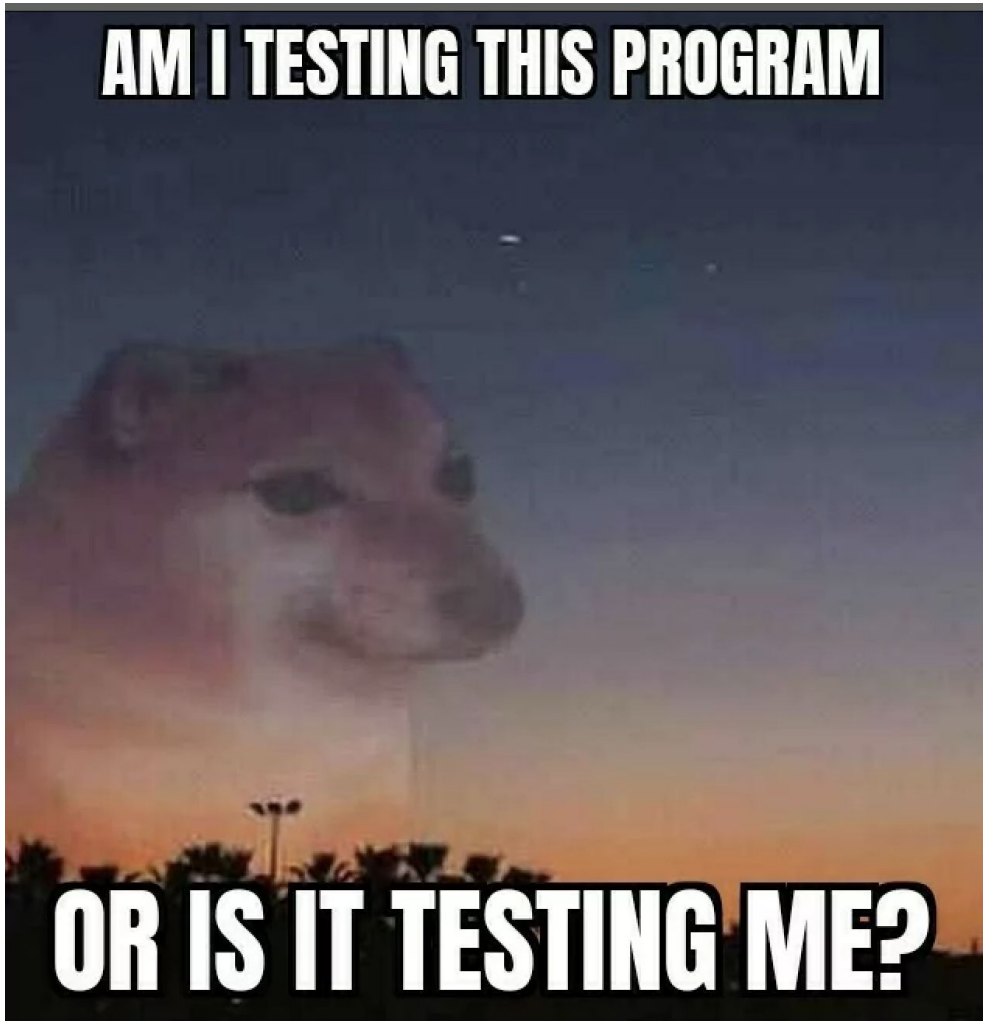
imgflip.com

Me every night !!




AM I BROKEN ?
I ACTUALLY ENJOY STAYING UP ALL NIGHT LONG CODING
quickmeme.com


I code better at night
11:00 PM
11:05 PM


AM I TESTING THIS PROGRAM
OR IS IT TESTING ME?


STAND BACK
WE'RE TRYING THIS IN PRODUCTION


ONE DOES NOT SIMPLY
ROLL THEIR OWN CRYPTO
imgflip.com











USER-DEFINED FUNCTIONS!

- Building blocks of code
- Set of executable statements (runs when called)

WHY?

- Easier to read (style)
- Easier to debug
- Easier to reuse, not repeat code
- Easier to organize as projects get bigger
- Modular

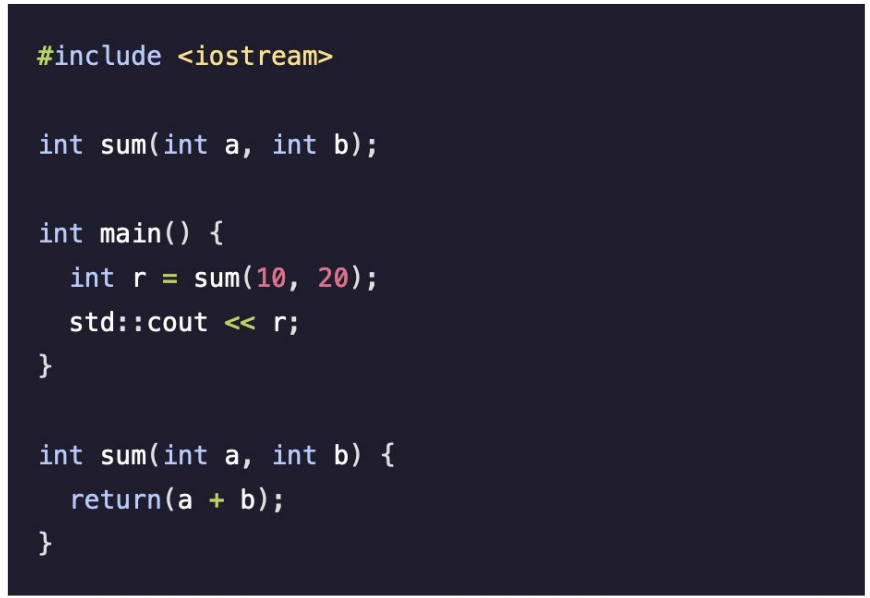






Function structure:

- **1. Declare**
  (above main)
- **2. Define**
  (below main)
- **3. Call**
  (inside main)



```cpp
#include <iostream>

int sum(int a, int b);

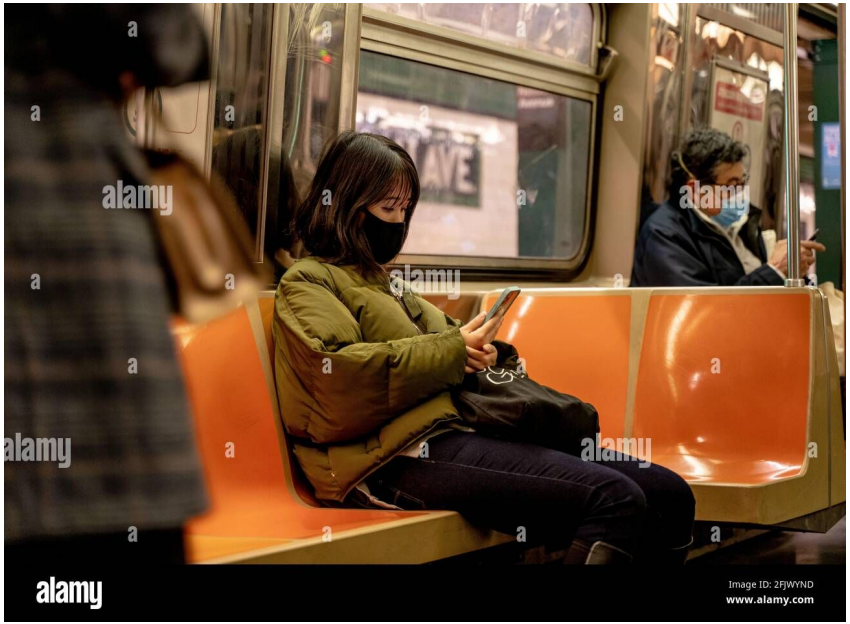
int main() {
  int r = sum(10, 20);
  std::cout << r;
}

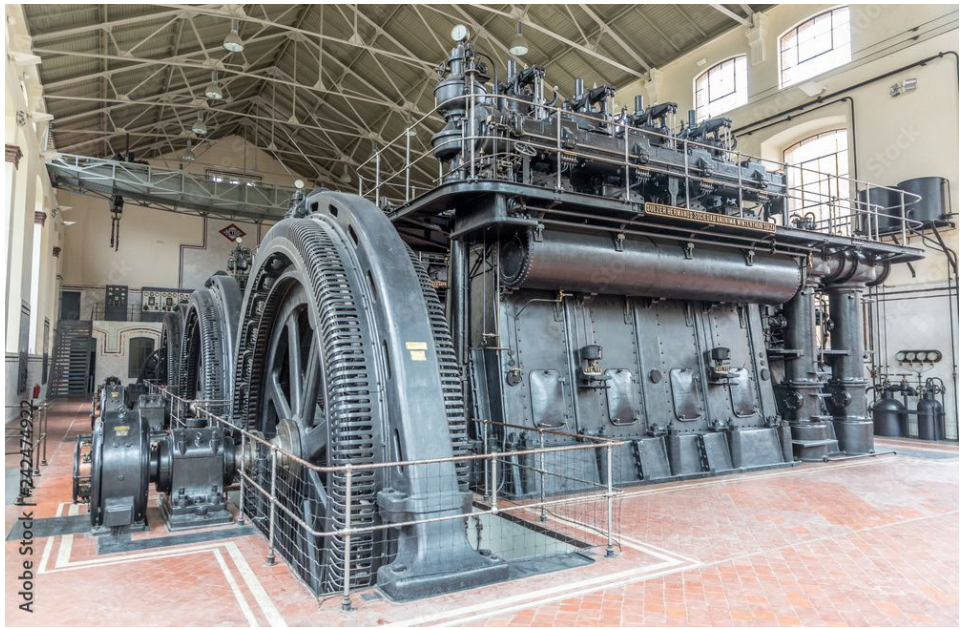
int sum(int a, int b) {
  return(a + b);
}
```

FUNCTIONS: abstraction

hiding or simplifying the **complexity** of some aspects of a system to make it easier for those using just **part** of it

farther **away** from the guts, the metal, computation, data, hardware, etc.

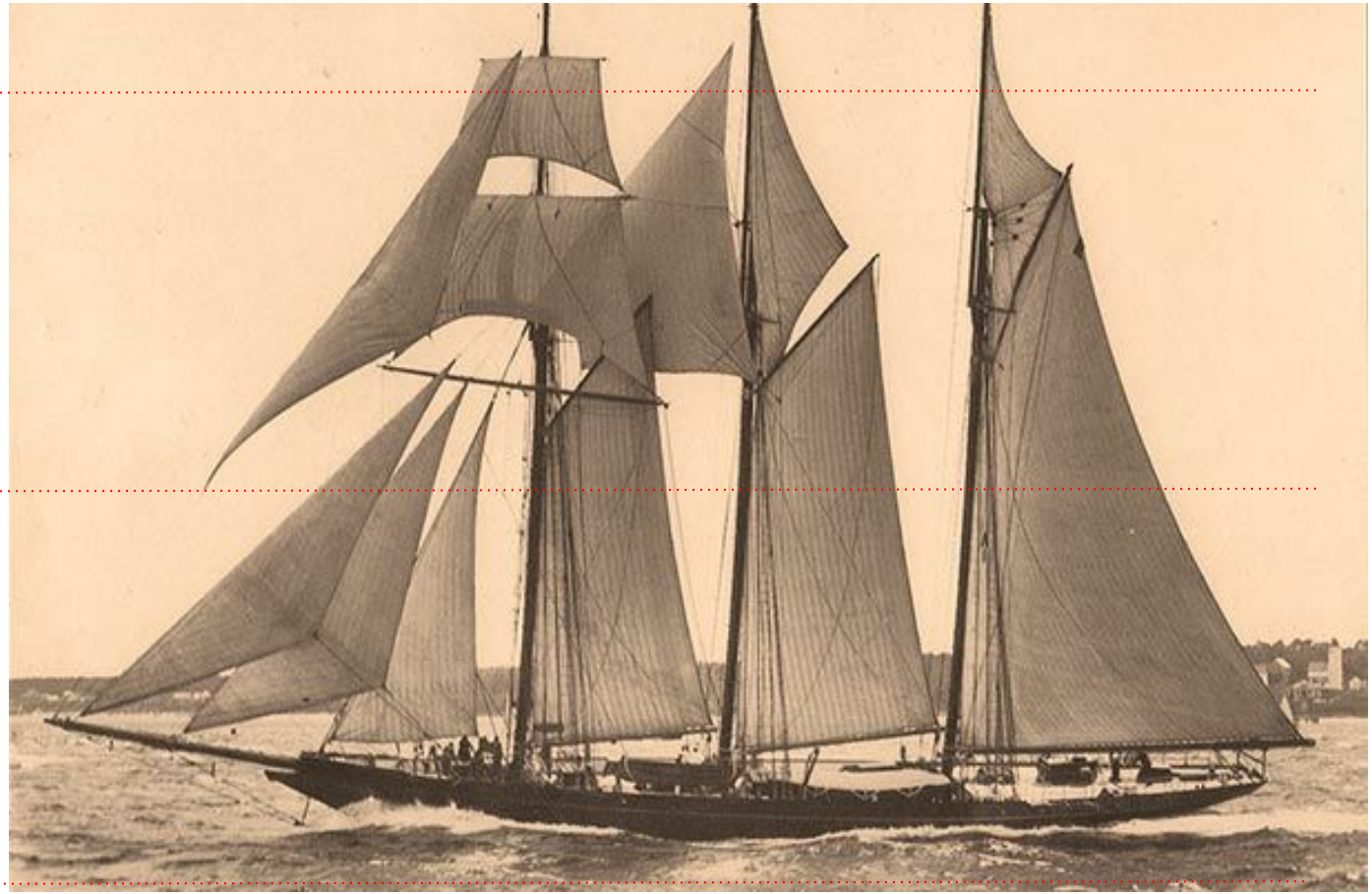




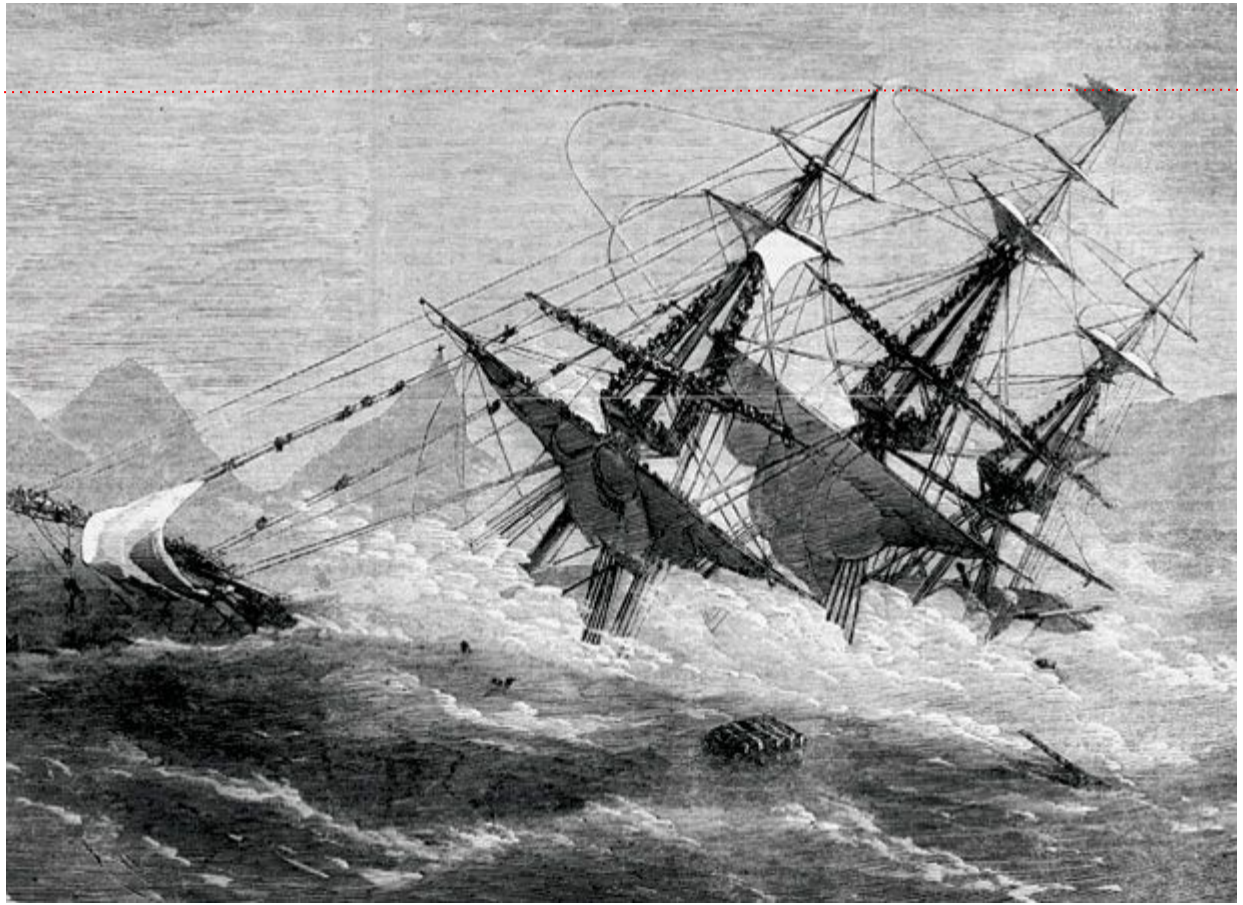
high-level

mid-level

low-level



high-level??



📣 Function declaration anatomy:

return type name(parameter list);

```
int myFunction(int var1, int var2);
```

📣 Function definition anatomy:

return type name(parameter list)

```
int myFunction(int var1, int var2)
{
// my code!
   return something
}
```

📣 Function call anatomy:

```
name(arguments);
```

```
myFunction(num1, num2);
```

📣 VOID function anatomy:

return type name(parameter list)

```
void sayHello(string name) {
// my code!
    cout << “hello, ” << name;
}
```

```cpp
#include <iostream>

// Declaring a function
void print();

int main() {
  print();
}

// Defining a function
void print() {
  std::cout << "Hello World!";
}
```

📣 MAIN function anatomy:

return type name(parameter list)
Return type = int
0 = exit code, “success”

```
int main() {
// my code!
   return 0
}
```

📣 MAIN function anatomy:

Other exit codes:

exit(1): It indicates abnormal termination of a program perhaps as a result a minor problem in the code

exit(2): It is similar to exit(1) but is displayed when the error occurred is a major one (not common)

exit(127): It indicates command not found

exit(133): It indicates that a program was aborted perhaps as a result of dividing an integer by zero



https://shorturl.at/UC4HZ

CS 103000
Prof. Madeline Blount

Week 9:
FUNCTIONS + ABSTRACTION

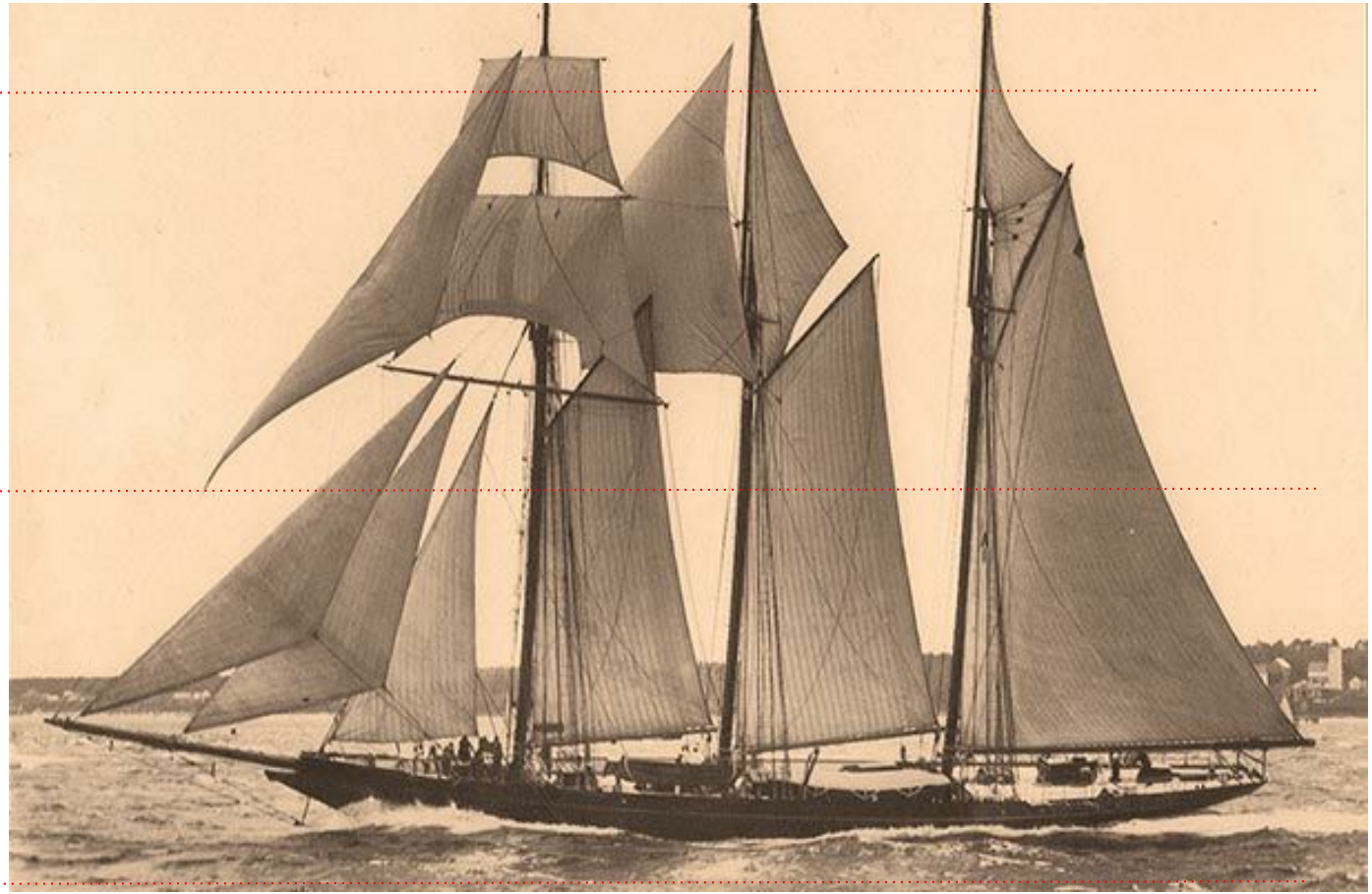


Dall-E 2: cats learning C++ in the forest on ‘90’s technology

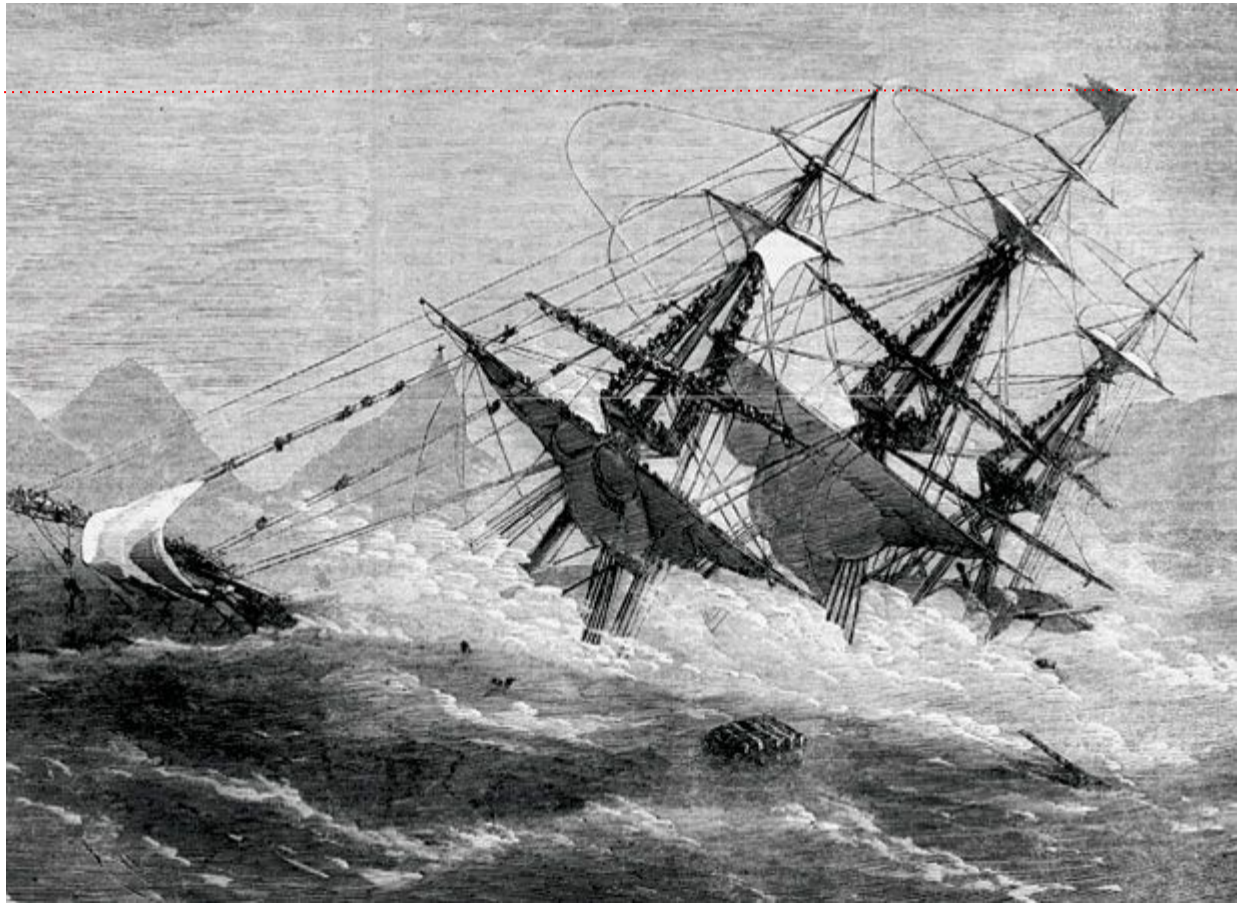
high-level

mid-level

low-level



high-level??



SCOPE


GLOBAL
everywhere!
local
only accessible in the function where they were created





```cpp
#include <iostream>

void print();

int i = 10;       // global variable

int main() {
  std::cout << i << "\n";
}

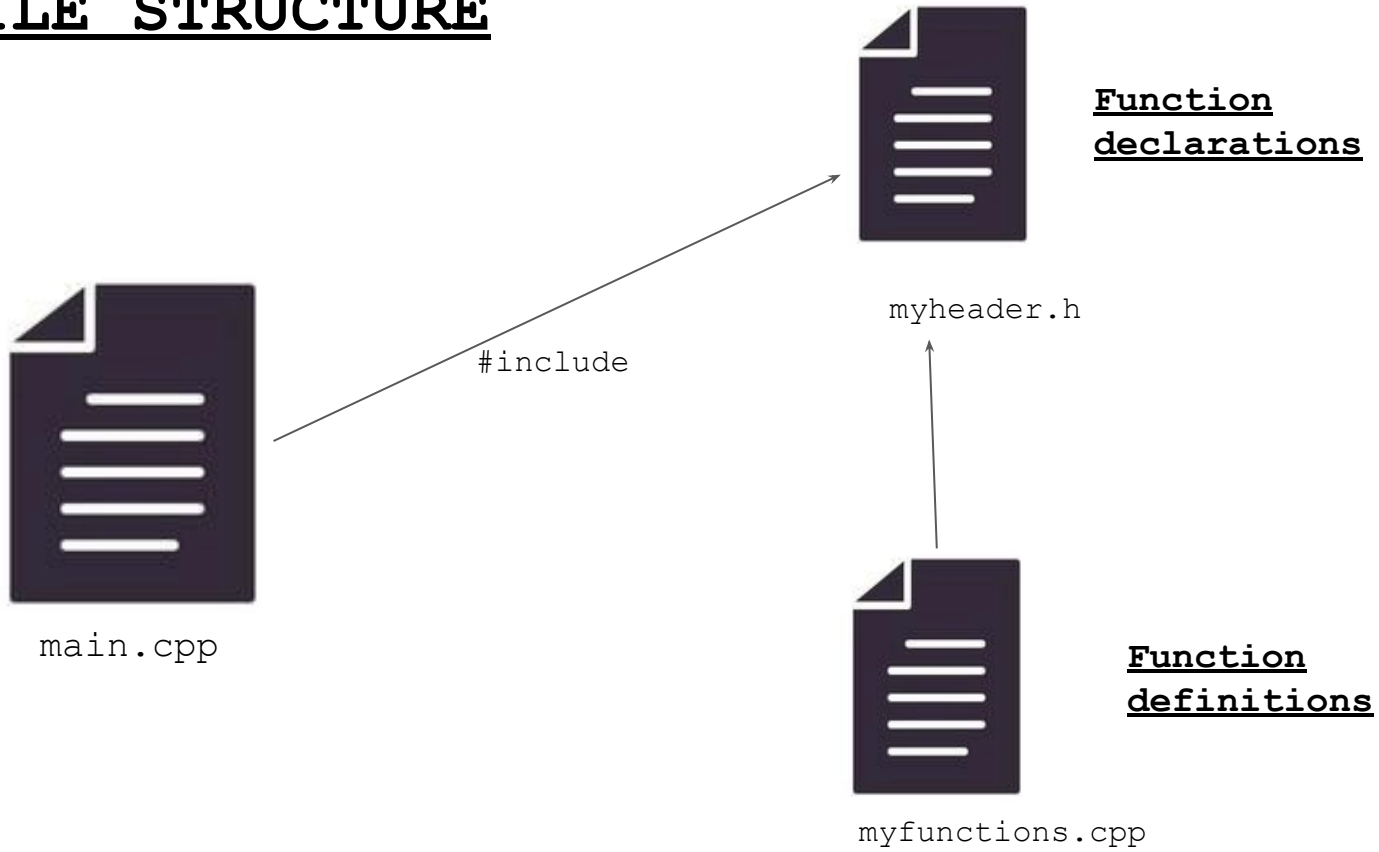
void print() {
  int j = 0;      // local variable
  i = 20;
  std::cout << i << "\n";
  std::cout << j << "\n";
}
```

"Function overload":

Functions can have the same name but handle different data types

```
int cubeNumber(int x);
double cubeNumber(double x);
```

FILE STRUCTURE



compile: `g++ main.cpp myfunctions.cpp`